Project Files and Folders

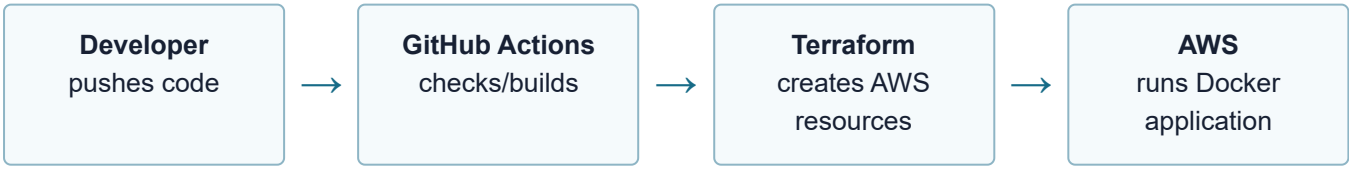
Simple explanation guide for the AWS Docker Deployment Platform

> **What this project does:** Terraform prepares an AWS network and application platform. Internet traffic enters through an Application Load Balancer. Docker application instances run inside private subnets and scale using an Auto Scaling Group. The application can use private RDS PostgreSQL and Redis services. GitHub Actions checks Terraform and can build/deploy an application after its source and Dockerfile are supplied.

1. Project structure

```
adlur/
├── .github/workflows/
│   ├── build-deploy.yml
│   └── terraform-check.yml
├── docs/
│   ├── architecture.md
│   ├── architecture.html
│   ├── architecture.pdf
│   ├── project-file-guide.html
│   └── runbook.md
├── evidence/
│   └── .gitkeep
├── infra/aws/
│   ├── templates/user-data.sh.tftpl
│   ├── .terraform.lock.hcl
│   ├── compute.tf
│   ├── data.tf
│   ├── main.tf
│   ├── outputs.tf
│   ├── terraform.tfvars.example
│   ├── variables.tf
│   └── versions.tf
├── scripts/check-health.ps1
├── .gitignore
├── Makefile
└── README.md
```

2. How the parts work together



The `docs` folder explains the design. The `infra/aws` folder contains the actual infrastructure code. The `.github/workflows` folder automates checks and deployment. The `scripts` folder contains a health-test helper. The `evidence` folder is reserved for real test results.

3. Root-level files

File	What it contains	Why it is used
`README.md`	Main project introduction, implemented features, deployment instructions, limitations and evidence checklist.	This is the first file a reviewer should read. It explains what the project does and how to use it.
`Makefile`	Short commands named `fmt`, `validate` and `check`.	It avoids typing long Terraform commands. For example, `make validate` initializes and validates Terraform.
`.gitignore`	A list of local or sensitive files that Git must not upload, such as Terraform state, plans and real variable files.	It prevents secrets, generated files and local Terraform state from being committed accidentally.

> **Important:** `terraform.tfvars` and `*.tfstate` must not be committed. They can contain account details and secret values.

4. GitHub Actions — `.github/workflows/`

GitHub automatically reads workflow files from this folder.

File	What it contains	Why it is used
`terraform-check.yml`	A job that checks Terraform formatting, downloads providers without configuring remote state, and validates the configuration.	It catches formatting and Terraform configuration errors on pull requests and pushes.
`build-deploy.yml`	The application delivery pipeline: application preflight, AWS OIDC login, Docker build, Trivy security scan, ECR push, digest lookup, Terraform plan and apply.	It supports “build once, deploy the same image digest.” If no root Dockerfile exists, application steps and deployment are intentionally skipped.

Why deployment may show “Skipped”

The assignment did not include the black-box application source or a Dockerfile. The workflow checks for a root `Dockerfile`. If it is absent, `app_available=false`; the AWS build and deploy stages are skipped instead of pretending that an application was deployed.

5. Documentation — `docs/`

File	What it contains	Why it is used
`architecture.pdf`	The final five-page architecture submission with diagram, decisions, rejected alternatives, scaling, operations, top five risks and roadmap.	This is the document to submit or show to the reviewer.
`architecture.html`	The styled, editable source used to generate the architecture PDF.	Edit this when the formatted PDF design or wording must change.
`architecture.md`	A Markdown version with more detailed architecture explanations.	Easy to read directly in GitHub and easy to edit without special software.
`runbook.md`	Operational steps for deployment failure, high latency, queue backlog, database incidents, restore testing and NAT/AZ failure.	It tells an operator what to check and do during common incidents.
`project-file-guide.html`	The editable source of this simple project guide.	It can be updated and printed again whenever project files change.
`project-file-guide.pdf`	This guide in portable PDF format.	It is easy to share and explain during the review.

6. Evidence — `evidence/`

File/folder	What it contains	Why it is used
`evidence/`	A reserved location for sanitized deployment, load-test, rollback and restore-test results.	The assignment asks for real evidence. Only actual test output should be saved here.
`.gitkeep`	An empty placeholder file.	Git does not track empty folders, so this keeps the evidence folder in the repository until evidence is added.

> **Do not create fake evidence.** Add logs or screenshots only after running the deployment or test. Remove passwords, tokens and customer data first.

7. AWS infrastructure — `infra/aws/`

Terraform reads all `.tf` files in this directory together. File names organize the code for humans; they do not change Terraform execution order.

File	What it contains	Why it is used
`versions.tf`	Terraform version requirement, AWS and Random provider versions, S3 remote-state backend declaration, AWS region and common tags.	It makes tool/provider requirements clear and prepares safe shared remote state.
`variables.tf`	Inputs such as project name, environment, region, CIDR, image URI, instance sizes, ASG limits, database size and Redis size.	The same Terraform code can be reused for development, staging and production without rewriting resources.
`terraform.tfvars.example`	Example values for every important variable.	Copy it to `terraform.tfvars` and replace the examples. The real file stays ignored by Git.
`main.tf`	Availability Zones, VPC, two public subnets, two private app subnets, two private DB subnets, IGW, NAT, route tables and security groups.	This creates the network foundation and controls which components can communicate.
`compute.tf`	Amazon Linux AMI lookup, EC2 IAM role/profile, ECR/SSM/secret permissions, ALB, target group, listener, launch template, ASG and scaling policy.	This runs Docker application instances, distributes traffic, replaces unhealthy instances and scales capacity.
`data.tf`	Random database password, Secrets Manager secret, RDS subnet group/PostgreSQL instance, Redis subnet group and replication group.	This creates private managed data services and avoids hardcoding a database password.
`outputs.tf`	Useful results such as ALB DNS name, VPC ID, private subnet IDs and database secret ARN.	It displays values needed after deployment or by other automation.
`.terraform.lock.hcl`	Exact provider versions and checksums selected by `terraform init`.	It makes provider installation repeatable and should normally be committed.

8. EC2 startup template — `infra/aws/templates/`

File	What it contains	Why it is used
`user-data.sh.tftpl`	A startup shell template that installs Docker/AWS CLI, reads database credentials from Secrets Manager, logs in to ECR and starts the application container on port 8080.	Each new Auto Scaling instance configures itself automatically without console or SSH steps.

9. Helper script — `scripts/`

File	What it contains	Why it is used
`check-health.ps1`	A PowerShell script that repeatedly requests `http://ALB-DNS/healthz` and reports success or failure.	It provides a simple post-deployment health check and can be used while collecting real evidence.

10. Tool-managed folders

Folder	What it contains	What you should do
`.git/`	Git history, branches, commit objects, configuration and internal metadata.	Do not edit its files manually. Use Git commands such as `git status`, `git add` and `git commit`.
`infra/aws/.terraform/`	Downloaded Terraform providers and local initialization data.	Do not commit or edit it. Terraform recreates it with `terraform init`.

11. Simple deployment sequence

1. Copy `terraform.tfvars.example` to `terraform.tfvars` and enter correct values.
2. Configure the S3 state bucket, DynamoDB lock table and AWS credentials.
3. Run `terraform init`, `terraform plan` and `terraform apply`.
4. Terraform creates networking, security, ALB, Auto Scaling, RDS and Redis.
5. When real application source and a root Dockerfile are available, the build workflow scans and pushes the image to ECR.
6. The workflow deploys the image digest; ASG replaces instances gradually and the ALB sends traffic only to healthy targets.
7. Run `scripts/check-health.ps1` with the ALB DNS name and save only real, sanitized evidence.

12. What is implemented and what is not

Implemented in code	Documented/proposed only
VPC, subnets, IGW, NAT, routes, security groups, ALB, EC2 ASG, Docker startup, RDS, Redis, Secrets Manager, Terraform checks and deployment template.	Kubernetes, second cloud, GitOps, metric-gated canary, worker-specific command/scaling, tenant fleet controller, and executed load/rollback/restore evidence.

> **Best review explanation:** “This project implements the AWS infrastructure that I understand and can defend. Advanced assignment requirements are clearly described as future work instead of being falsely marked as completed.”